

AEROBIC.SPACE

Выполнили: Дикие красавчики

Постановка задачи

Входные данные:

referees.csv → 200+ судей

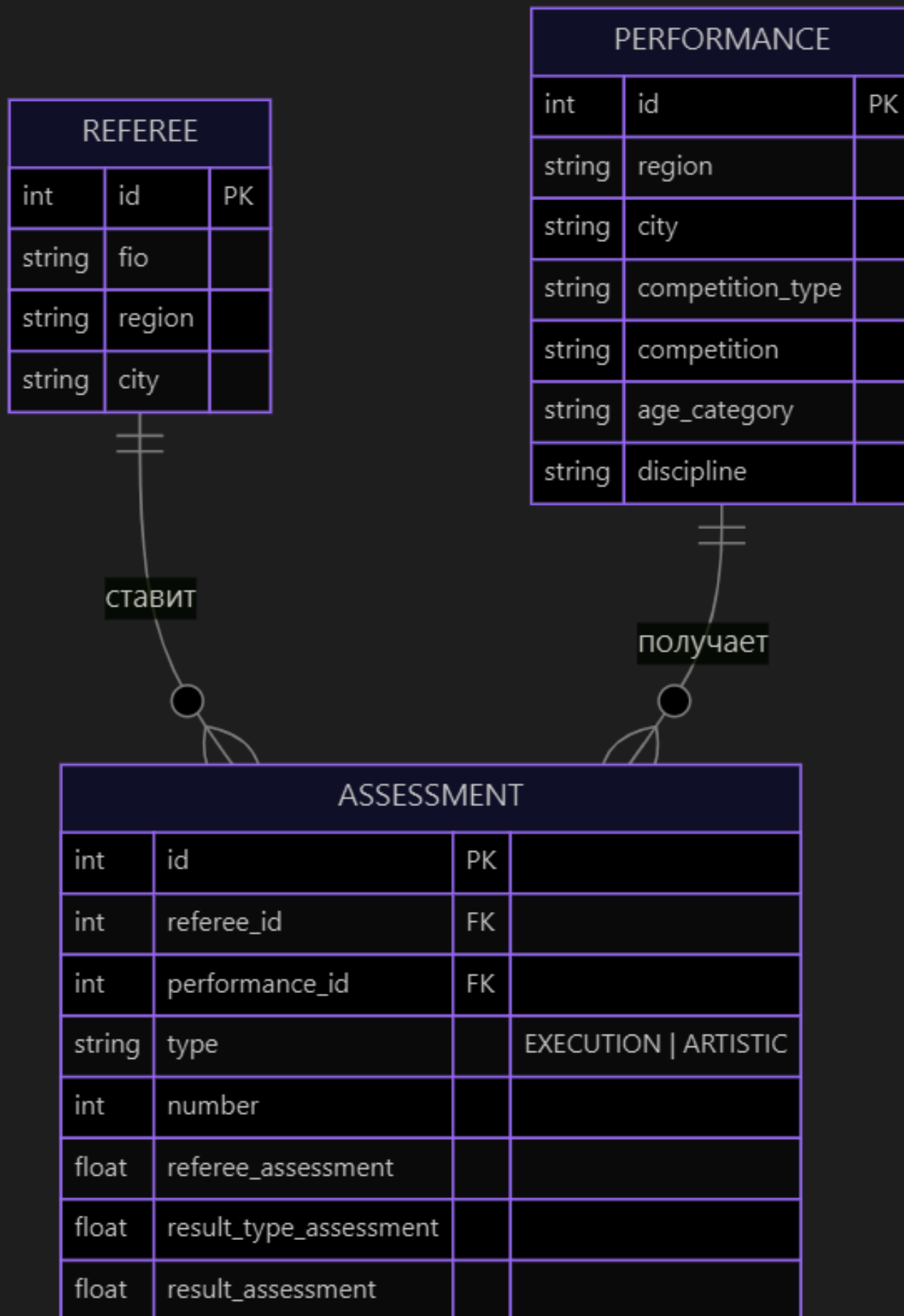
performances.csv → выступления (соревнование, регион, дисциплина)

assessments.csv → оценки судей (referee × performance × type)

Объём: десятки тысяч строк

Формат: CSV без заголовков, UTF-8 BOM

Задача: вычислить метрики качества судейства



Алгоритм #1: Точность

```
def get_threshold(result_score: float) -> float:
    if result_score >= 8.0:
        return 0.3
    elif result_score >= 7.0:
        return 0.4
    elif result_score >= 6.0:
        return 0.5
    else:
        return 0.6

def classify_accuracy(referee_score: float, result_score: float) -> str:
    deviation = abs(referee_score - result_score)
    if deviation == 0:
        return "bullseye"
    elif deviation <= get_threshold(result_score):
        return "acceptable"
    else:
        return "serious"

def calc_accuracy_percent(assessments: list) -> float:
    if len(assessments) == 0:
        return 0.0

    counter = 0
    for a in assessments:
        label = classify_accuracy(a.referee_assessment, a.result_type_assessment)
        if label != "serious":
            counter += 1

    return counter / len(assessments) * 100
```

ref=8.2, result=8

threshold=0.3

d=0.2 → acceptable

```
bias = mean(deviation_others) - mean(deviation_own)
```

own = выступления "своего" региона/города

└─ RUSSIA → match по region

└─ REGION → match по city

$bias > 0$ судья мягче к своим

$bias < 0$ строже к своим

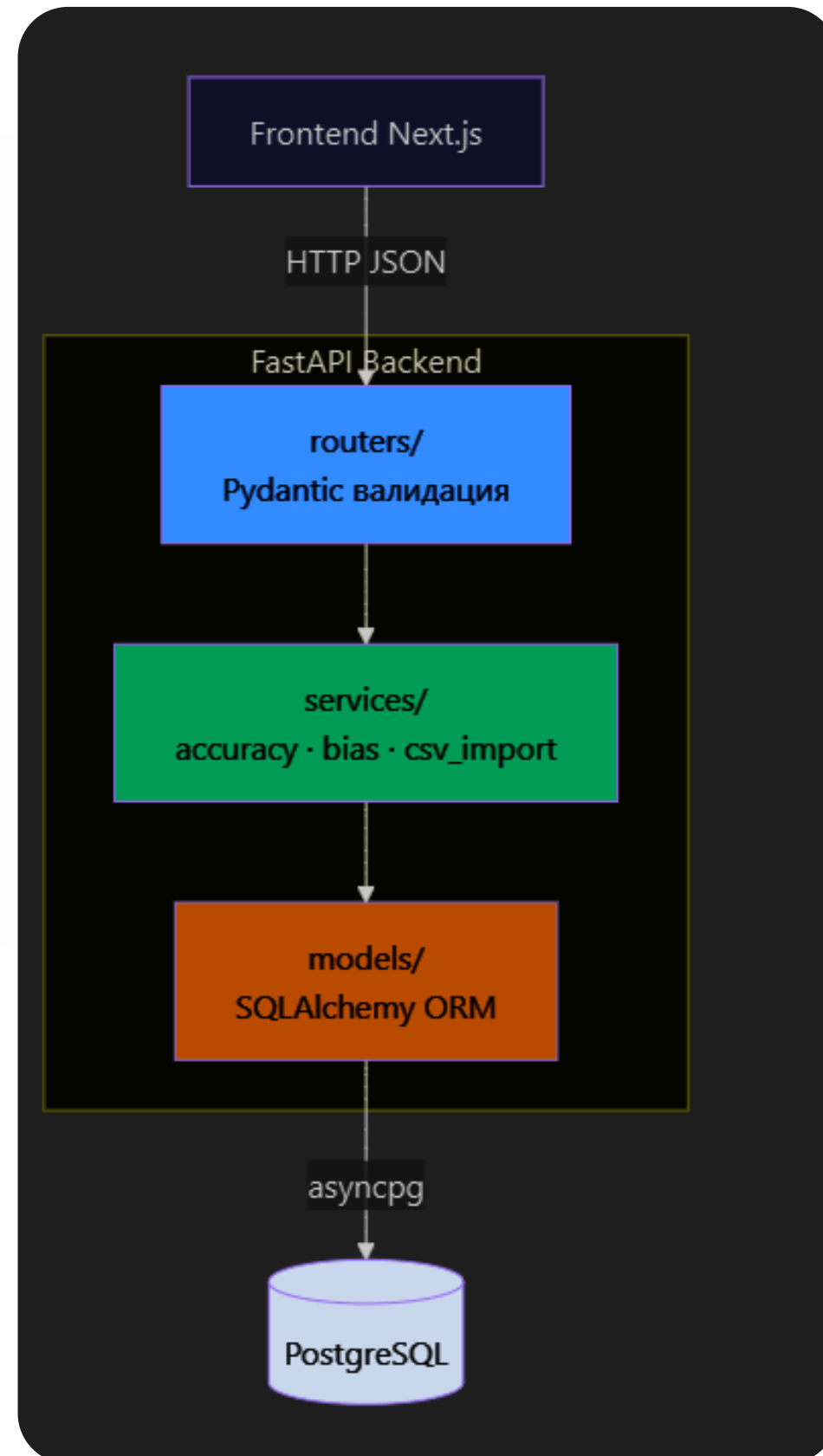
$|bias| > \text{порог} \rightarrow \text{red flag для жюри}$

Алгоритм #2:

Предвзятость



Backend архитектура



```
app/  
├ routers/      ← HTTP, Pydantic-валидация  
├ services/     ← accuracy, bias, csv_import  
├ models/       ← SQLAlchemy ORM (3 таблицы)  
├ schemas/      ← Pydantic (контракт с фронтом)  
├ constants.py  ← JudgeType, CompetitionType  
└ database.py   ← async engine
```

8 эндпоинтов · async везде · Pydantic Settings

Оптимизация

Фиксы перфомансов

✗ N+1 в /competitions/{name}

→ 1 запрос + группировка в Python

✗ N+1 в /referees/ (JOIN на каждого судью)

→ 1 JOIN-запрос с in_(referee_ids)

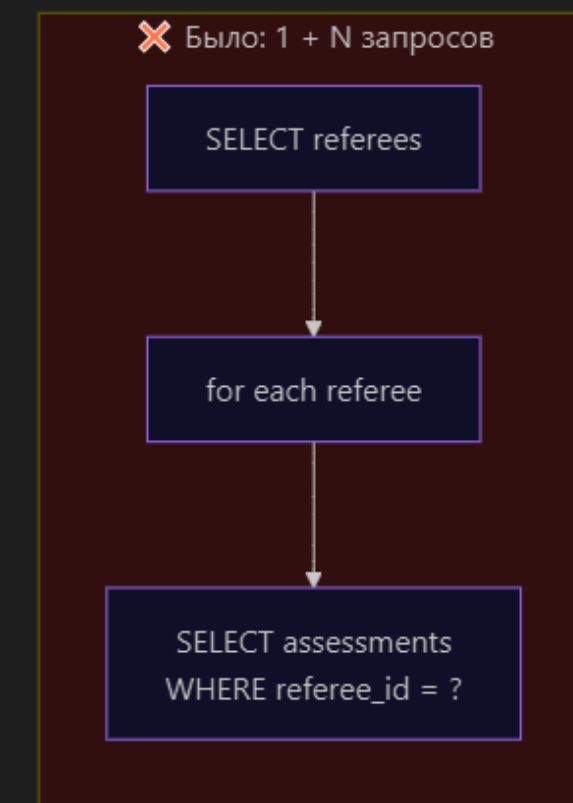
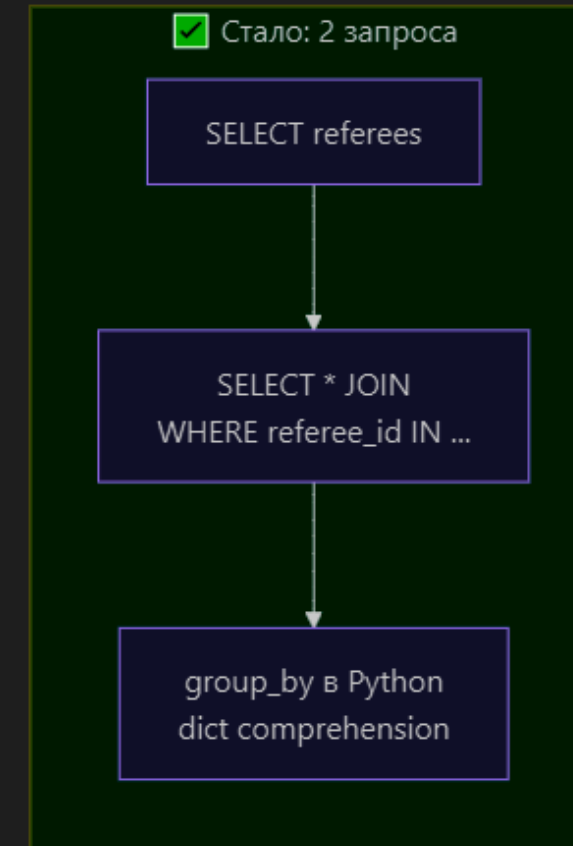
✗ FullScan на WHERE competition = ...

→ indexes на 8 полях

✗ Дубль кода: 4× повтор calc accuracy

→ calc_exec_art_accuracy()

→ JudgeType.EXECUTION вместо
"EXECUTION"



Тесты

pytest · 28 tests · 100% green · 2.6s

test_accuracy.py 11 pure unit (dataclass
fakes)

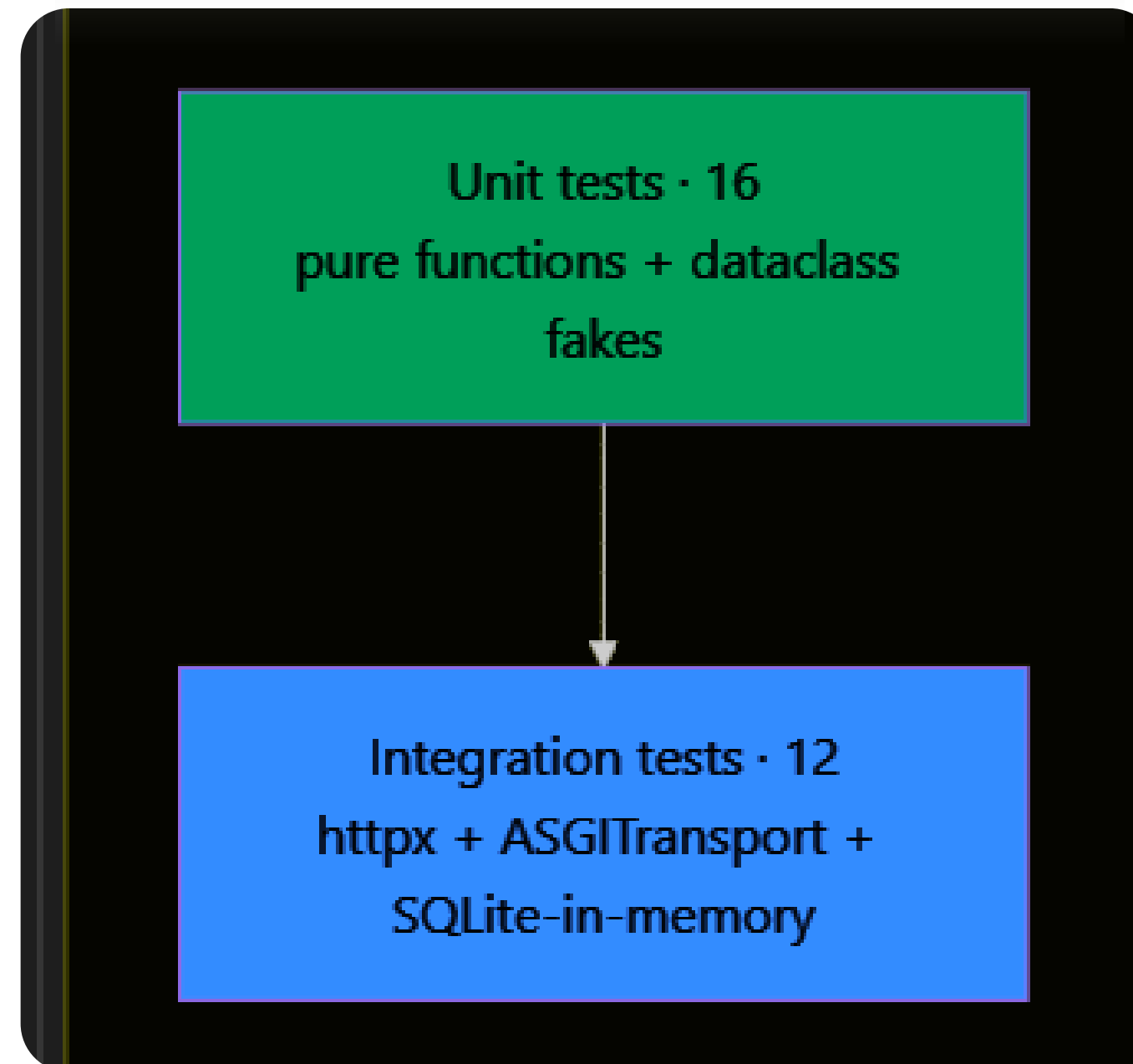
test_bias.py 5 pure unit

test_referees.py 6 integration (httpx + asgi)

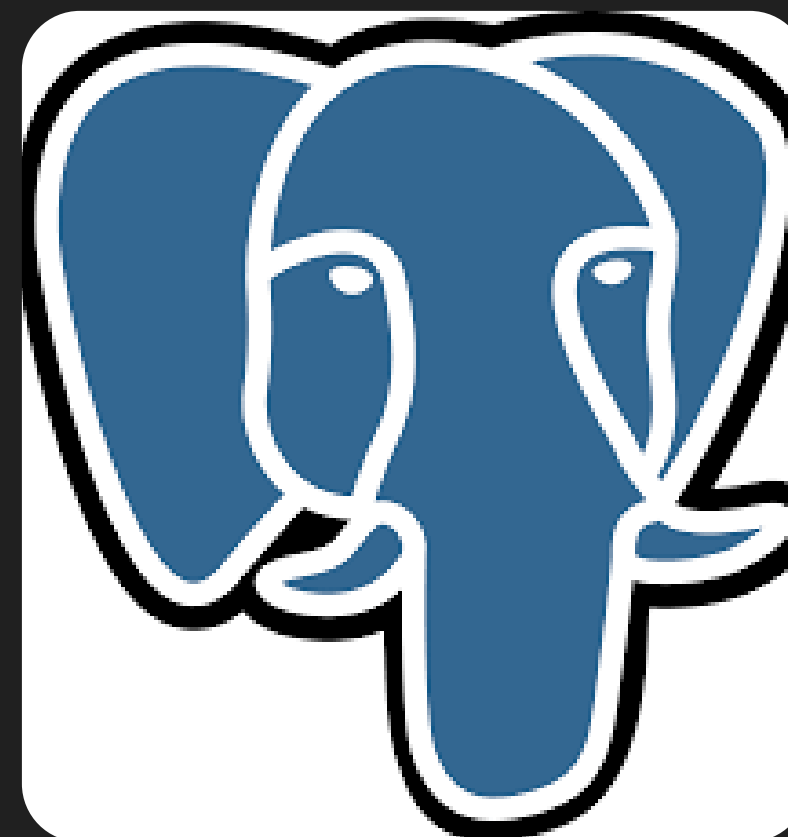
test_upload.py 6 integration

Изоляция: SQLite-in-memory + aiosqlite

Каждый тест → чистая схема, fixture-
overrides



Стек и инфраструктура



Backend

Python 3.11 · FastAPI · SQLAlchemy 2.0 (async)
asynpg · Pydantic v2 · pytest

Database

PostgreSQL 16 · 8 индексов · NOT NULL

Frontend

Next.js 16 · React 19 · TypeScript · Tailwind 4

Infra

Docker Compose · .env · CORS lockdown

Метрики

8 эндпоинтов API
28 тестов (pytest)
3 таблицы PostgreSQL
8 индексов
2 ключевых алгоритма
~600 строк бизнес-логики
~100% покрытие services/



**Спасибо за
внимание!**